

```

#include <stdio.h>

#define MAX 10

typedef struct {
    char pid[5];
    int at, bt, rt, ct, tat, wt, q;
} Process;

Process p[MAX];
int n, tq = 2;

char gantt_p[100][5];
int gantt_t[100];
int g_index = 0;

void addGantt(char pid[], int time) {
    int i;
    for (i = 0; pid[i]; i++)
        gantt_p[g_index][i] = pid[i];
    gantt_p[g_index][i] = '\0';

    gantt_t[g_index] = time;
    g_index++;
}

int main() {
    int i, time = 0, completed = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("PID: ");
        scanf("%s", p[i].pid);
        printf("Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Queue (1=RR, 2=FCFS): ");
        scanf("%d", &p[i].q);
    }
}

```

```
    p[i].rt = p[i].bt;  
}
```

```
while (completed < n) {
```

```
    int executed = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        if (p[i].q == 1 && p[i].rt > 0 && p[i].at <= time) {
```

```
            executed = 1;
```

```
            if (p[i].rt > tq) {
```

```
                addGantt(p[i].pid, time);
```

```
                time += tq;
```

```
                p[i].rt -= tq;
```

```
            } else {
```

```
                addGantt(p[i].pid, time);
```

```
                time += p[i].rt;
```

```
                p[i].rt = 0;
```

```
                p[i].ct = time;
```

```
                completed++;
```

```
            }
```

```
        }
```

```
    }
```

```
if (!executed) {
```

```
    for (i = 0; i < n; i++) {
```

```
        if (p[i].q == 2 && p[i].rt > 0 && p[i].at <= time) {
```

```
            addGantt(p[i].pid, time);
```

```
            time += p[i].rt;
```

```
            p[i].rt = 0;
```

```
            p[i].ct = time;
```

```
            completed++;
```

```
            executed = 1;
```

```
            break;
```

```
        }
```

```
    }
```

```
}
```

```
if (!executed)
```

```

        time++;
    }

    float avg_tat = 0, avg_wt = 0;

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for (i = 0; i < n; i++) {
        p[i].tat = p[i].ct - p[i].at;
        p[i].wt = p[i].tat - p[i].bt;

        avg_tat += p[i].tat;
        avg_wt += p[i].wt;

        printf("%s\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt,
            p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time = %.2f", avg_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", avg_wt / n);

    printf("\nGantt Chart:\n");

    for (i = 0; i < g_index; i++)
        printf("| %s ", gantt_p[i]);
    printf("\n");

    for (i = 0; i < g_index; i++)
        printf("%d ", gantt_t[i]);

    printf("%d\n", time);

    return 0;
}

```

```
C:\Users\Admin\Desktop\IW/ X + v
Enter number of processes: 2

Process 1
PID: P1
Arrival Time: 3
Burst Time: 5
Queue (1=RR, 2=FCFS): 2

Process 2
PID: P2
Arrival Time: 2
Burst Time: 5
Queue (1=RR, 2=FCFS): 1

Process AT    BT    CT    TAT    WT
P1    3    5    12    9    4
P2    2    5    7    5    0

Average Turnaround Time = 7.00
Average Waiting Time = 2.00

Gantt Chart:
| P2 | P2 | P2 | P1 |
2   4   6   7   12

Process returned 0 (0x0)  execution time : 85.428 s
Press any key to continue.
|
```